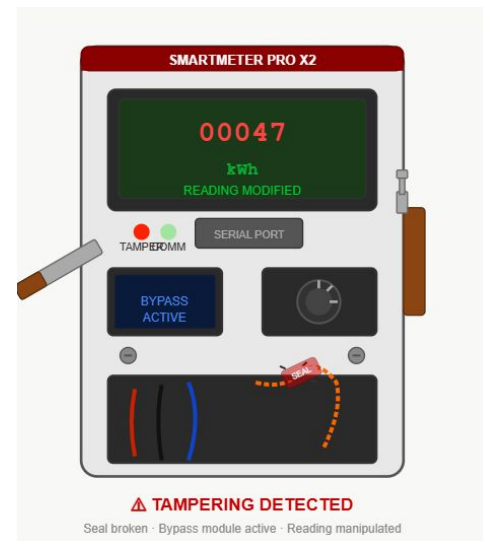


ELECTRICITY THEFT DETECTION IN SMART GRIDS USING MACHINE LEARNING

Mentor: Dr. Pradnya Khandeparker

GROUP 11

Vedant Chaugule - A007
Rajeshwari Majumdar - A033
Sarrah Pittalwala - A044
Jigar Rambhiya - A046
Symprose Remedios - A049



Contents

01

Abstract and Problem Statement

02

Aim & Objectives

03

Dataset — TDD2022

04

Methodology

05

Meta-Learner Selection

06

Results & Analysis

07

Limitations & Future Scope

Abstract & Problem Statement

\$96BAnnual NTL
worldwide**560K+**TDD2022 dataset
instances**2 Scenarios**P7C (7-class)
P6C (6-class)**3 Base +
1 Meta**

Ensemble architecture

Problem Statement

Electricity theft detection in smart grids is a complex multi-class classification problem characterized by high-dimensional consumption data, severe class imbalance, and overlapping patterns between normal and fraudulent usage.

There is a need to develop a robust, data-driven model capable of accurately identifying diverse theft behaviors while effectively handling imbalance and complex consumption dynamics.

Abstract

This study addresses electricity theft detection in smart grid systems using machine learning on large-scale consumption data.

A stacking-based ensemble model is implemented to enhance classification performance by optimally combining multiple base learners, outperforming traditional approaches.

Aim & Objectives

Aim

The aim of this study is to develop a machine learning-based decision support system for automated electricity theft detection in smart grid environments, leveraging consumer energy consumption patterns from the TDD2022 dataset to accurately classify normal and fraudulent usage across multiple theft categories.

Study Objectives

- 1 To develop a machine learning-based decision support system for automated electricity theft detection in smart grid environments using the TDD2022 dataset.
- 2 To implement and evaluate the system across two experimental scenarios — P7C (7-class classification including Theft 6) and P6C (6-class classification excluding Theft 6)
- 3 To extend the baseline system by introducing a stacking ensemble that replaces soft voting with a meta-learner trained on out-of-fold base model predictions, and to evaluate its classification performance across both experimental scenarios.

Dataset: TDD2022

560,640

Total instances

16

Consumer
types

11

Input
features

6

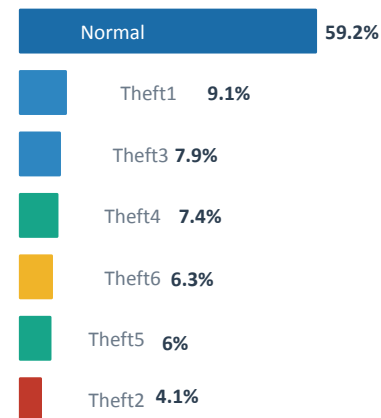
Synthetic
theft types

59.2%

Normal class
share

Theft Type	Mechanism	Instances
Theft 1	Hourly values $\times$ constant $\alpha \in [0.1, 0.8]$	51,083
Theft 2	Zero-consumption window ≥ 4 hours	22,958
Theft 3	Per-hour random reduction factor	44,349
Theft 4	Replace with random fraction of daily mean	41,460
Theft 5	All readings = daily mean (flat profile)	33,553
Theft 6	Time-reversed hours (hardest to detect)	35,413

Class Distribution



Synthetic Theft Pattern Generation

Theft Type	Formula	Parameters	Meaning
Theft ₁	$\text{Theft}_1(x_i) = \alpha x_i$	$\alpha = \text{random}(0.1, 0.8)$	Uniform reduction
Theft ₂	$\text{Theft}_2(x_i) = \beta_i x_i$	$\beta_i = \begin{cases} 0, & t_{start} < i < t_{end} \\ 1, & \text{otherwise} \end{cases}$	Zero in time window
Theft ₃	$\text{Theft}_3(x_i) = \gamma_i x_i$	$\gamma_i = \text{random}(0.1, 0.8)$	Random scaling
Theft ₄	$\text{Theft}_4(x_i) = \gamma_i \cdot \text{mean}(X)$	$\gamma_i = \text{random}(0.1, 0.8)$	Uses avg fraction
Theft ₅	$\text{Theft}_5(x_i) = \text{mean}(X)$	—	Constant usage
Theft ₆	$\text{Theft}_6(x_i) = x_{24-i}$	—	Reversed pattern

Dataset: TDD2022

Electricity:Facility [kW]/Hourly	Fans:Electricity [kW]/Hourly	Cooling:Electricity [kW]/Hourly	Heating:Electricity [kW]/Hourly	InteriorLights:Electricity [kW]/Hourly	InteriorEquipment:Electricity [kW]/Hourly	Gas:Facility [kW]/Hourly	Heating:Gas [kW]/Hourly	InteriorEquipment:Gas [kW]/Hourly	Water Heater:WaterSystems:Gas [kW]/Hourly	Class	thft
22.023597732	3.586220833	0	0	4.58992533	8.1892	136.5595007	123.9990759	3.33988	9.246946778	FullServiceRestaurant	Normal
14.64975748	0	0	0	1.52997511	7.4902	3.35988	0	3.33988	0.02	FullServiceRestaurant	Normal
14.66956666	0	0	0	1.52997511	7.4902	3.35988	0	3.33988	0.02	FullServiceRestaurant	Normal
14.67780801	0	0	0	1.52997511	7.4902	3.931931506	0	3.33988	0.592051506	FullServiceRestaurant	Normal
14.8247939	0	0	0	1.52997511	7.4902	3.35988	0	3.33988	0.02	FullServiceRestaurant	Normal
22.18264867	3.586220833	0.013197179	0	4.58992533	8.1892	130.5649421	121.6231847	8.3497	0.592057441	FullServiceRestaurant	Normal
38.13184509	3.586220833	0.007371036	0	9.179850659	19.4245	140.4354538	117.967915	16.6994	5.768138785	FullServiceRestaurant	Normal
45.59727804	3.586220833	0.007450454	0	9.179850659	26.9147	135.4560171	105.2134576	25.0491	5.193459464	FullServiceRestaurant	Normal
45.60364653	3.586220833	0	0	9.179850659	26.9147	145.5750769	106.4078096	33.3988	5.768467288	FullServiceRestaurant	Normal
38.11376101	3.586220833	0	0	9.179850659	19.4245	147.6959009	116.8790141	25.0491	5.767786783	FullServiceRestaurant	Normal
42.4352012	3.586220833	0.008012798	0	9.179850659	26.9147	177.5022984	117.6999854	41.7485	18.05381301	FullServiceRestaurant	Normal
42.1437099	3.586220833	0	0	9.179850659	26.9147	176.3227494	115.9380379	41.7485	18.63621145	FullServiceRestaurant	Normal
41.41962073	3.586220833	0	0	9.179850659	26.9147	167.5286575	111.2660792	41.7485	14.51407826	FullServiceRestaurant	Normal
33.89704606	3.586220833	0	0	9.179850659	19.4245	158.680649	110.7681901	33.3988	14.51365892	FullServiceRestaurant	Normal
33.89764389	3.586220833	0	0	9.179850659	19.4245	153.2839968	117.239112	25.0491	10.9957848	FullServiceRestaurant	Normal
36.00133359	3.586220833	0	0	9.179850659	19.4245	167.3348971	122.9381038	33.3988	10.99799331	FullServiceRestaurant	Normal
45.54644716	3.586220833	0.005751574	0	9.179850659	26.9147	185.4351957	124.3394303	50.0982	10.99756538	FullServiceRestaurant	Normal
45.55135973	3.586220833	0	0	9.179850659	26.9147	190.0373937	125.4255895	50.0982	14.51360419	FullServiceRestaurant	Normal
45.55679431	3.586220833	0	0	9.179850659	26.9147	193.6670575	124.9314663	50.0982	18.63739126	FullServiceRestaurant	Normal
38.19976737	3.586220833	0	0	9.179850659	19.4245	172.867008	121.4151244	33.3988	18.0530836	FullServiceRestaurant	Normal
38.18202589	3.586220833	0	0	9.179850659	19.4245	167.0888198	119.1771374	33.3988	14.5128284	FullServiceRestaurant	Normal
38.11299043	3.586220833	0	0	9.179850659	19.4245	174.8424699	131.7403991	25.0491	18.05297083	FullServiceRestaurant	Normal
38.00435973	3.586220833	0	0	9.179850659	19.4245	169.5218912	137.7268237	16.6994	15.09566753	FullServiceRestaurant	Normal
38.81510782	3.586220833	0	0	9.179850659	19.4245	153.8724665	138.017477	8.3497	7.505289457	FullServiceRestaurant	Normal
22.05446109	3.586220833	0	0	4.58992533	8.1892	158.0636088	147.7940647	3.33988	6.929716124	FullServiceRestaurant	Normal
14.61348165	0	0	0	1.52997511	7.4902	3.35988	0	3.33988	0.592092005	FullServiceRestaurant	Normal
14.63021954	0	0	0	1.52997511	7.4902	3.35988	0	3.33988	0.02	FullServiceRestaurant	Normal
14.64225703	0	0	0	1.52997511	7.4902	3.931926177	0	3.33988	0.592046177	FullServiceRestaurant	Normal
14.79326476	0	0	0	1.52997511	7.4902	3.35988	0	3.33988	0.02	FullServiceRestaurant	Normal
22.16287453	3.586220833	0.00586912	0	4.58992533	8.1892	165.2225549	156.2808027	8.3497	0.59205212	FullServiceRestaurant	Normal
38.12403843	3.586220833	0.006969506	0	9.179850659	19.4245	171.638704	149.1712159	16.6994	5.768088074	FullServiceRestaurant	Normal
45.59130406	3.586220833	0.008684217	0	9.179850659	26.9147	189.5961268	142.9299061	25.0491	21.67112069	FullServiceRestaurant	Normal
45.60018698	3.586220833	0.002049974	0	9.179850659	26.9147	192.4531198	142.2299061	33.3988	19.83401869	FullServiceRestaurant	Normal
38.11356489	3.586220833	0	0	9.179850659	19.4245	170.9660884	137.6328709	0	16.28411709	FullServiceRestaurant	Normal
42.42681181	3.586220833	0.004028388	0	9.179850659	26.9147	195.3161303	136.4734869	41.7485	15.09414337	FullServiceRestaurant	Normal
42.13807956	3.586220833	0	0	9.179850659	26.9147	185.9901069	127.962155	41.7485	16.27935186	FullServiceRestaurant	Normal
41.41885802	3.586220833	0	0	9.179850659	26.9147	177.9531982	117.6809183	41.7485	14.51376999	FullServiceRestaurant	Normal
38.89487678	3.586220833	0	0	9.179850659	19.4245	168.9772849	122.8272075	33.3988	12.75127732	FullServiceRestaurant	Normal
38.89353409	3.586220833	0	0	9.179850659	19.4245	162.3689302	126.332769	25.0491	10.99655325	FullServiceRestaurant	Normal
35.99896527	3.586220833	0	0	9.179850659	19.4245	134.3175435	129.9205418	33.3988	10.99820167	FullServiceRestaurant	Normal
45.54288297	3.586220833	0.0057091	0	9.179850659	26.9147	192.6958765	131.5999492	50.0982	10.9977273	FullServiceRestaurant	Normal
45.55042294	3.586220833	0.002341001	0	9.179850659	26.9147	190.2888654	125.0948665	50.0982	15.09579892	FullServiceRestaurant	Normal
45.55970584	3.586220833	0.002424443	0	9.179850659	26.9147	190.8978942	120.9633998	50.0982	19.83329437	FullServiceRestaurant	Normal
38.19937629	3.586220833	0	0	9.179850659	19.4245	187.1560961	132.1385977	33.3988	21.61869838	FullServiceRestaurant	Normal
38.18017902	3.586220833	0	0	9.179850659	19.4245	191.5060019	139.4713271	33.3988	18.63587479	FullServiceRestaurant	Normal
38.11129237	3.586220833	0	0	9.179850659	19.4245	189.5554903	144.6730403	25.0491	19.83335007	FullServiceRestaurant	Normal
38.00343077	3.586220833	0	0	9.179850659	19.4245	178.0537288	145.073523	16.6994	16.28080574	FullServiceRestaurant	Normal
38.81433683	3.586220833	0	0	9.179850659	19.4245	162.3757213	144.7766441	8.3497	9.249977207	FullServiceRestaurant	Normal
22.055598	3.586220833	0.000951915	0	4.58992533	8.1892	153.1606649	142.3146474	3.33988	7.506137518	FullServiceRestaurant	Normal
14.62144499	0	0	0	1.52997511	7.4902	3.931929473	0	3.33988	0.592049473	FullServiceRestaurant	Normal

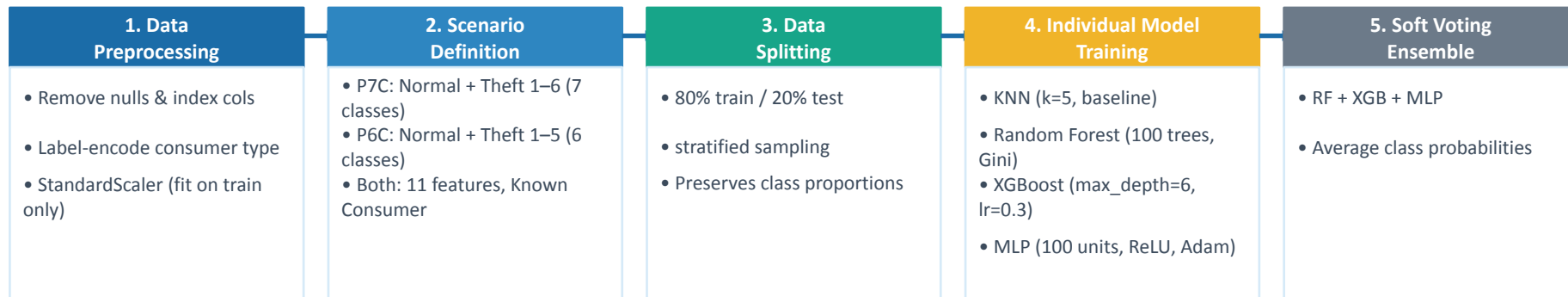
List of consumer types

SN*	Customer Type	SN*	Customer Type
1	FullServiceRestaurant	9	Warehouse
2	Hospital	10	SecondarySchool
3	LargeHotel	11	SmallHotel
4	LargeOffice	12	SmallOffice
5	MediumOffice	13	Stand-aloneRetail
6	MidriseApartment	14	StripMall
7	PrimarySchool	15	SuperMarket
8	OutPatient	16	QuickServiceRestaurant

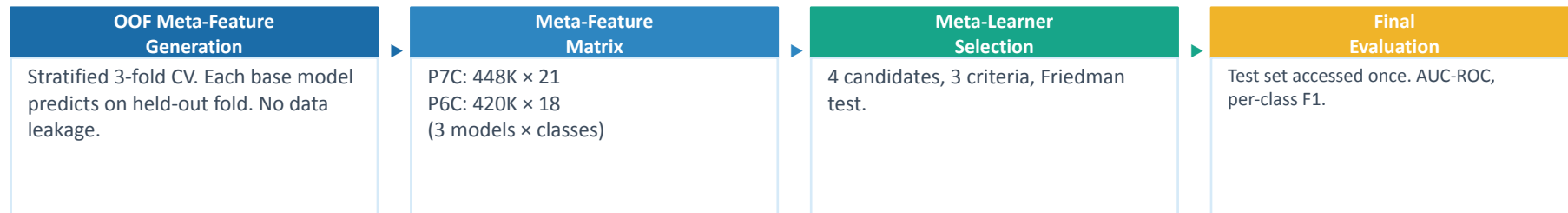
Features Information

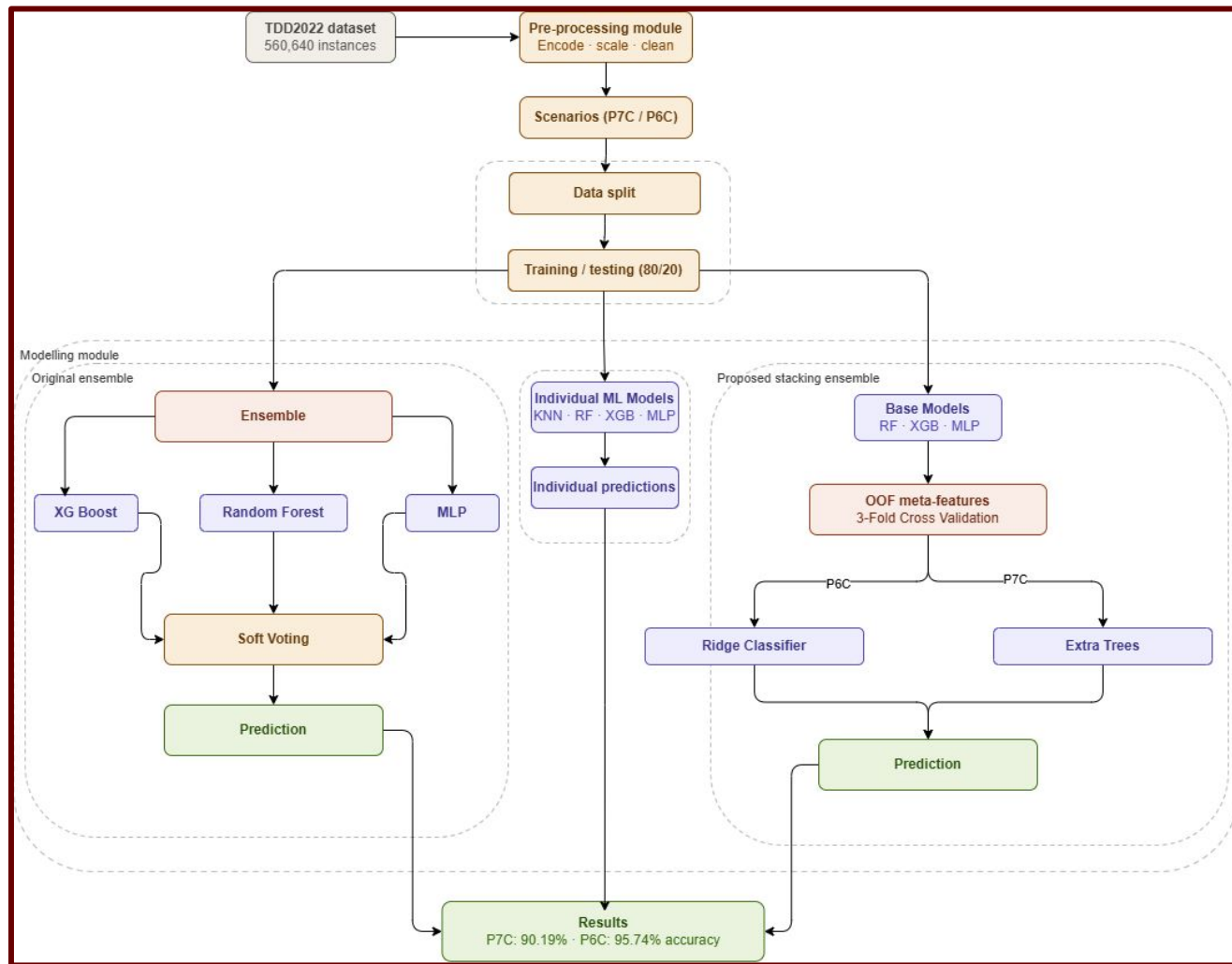
Feature Name	Type
Electricity:Facility [kW](Hourly)	Float
Fans:Electricity [kW](Hourly)	Float
Cooling:Electricity [kW](Hourly)	Float
Heating:Electricity [kW](Hourly)	Float
InteriorLights:Electricity [kW](Hourly)	Float
InteriorEquipment:Electricity [kW](Hourly)	Float
Gas:Facility [kW](Hourly)	Float
Heating:Gas [kW](Hourly)	Float
InteriorEquipment:Gas [kW](Hourly)	Float
WaterHeater:WaterSystems:Gas [kW](Hourly)	Float
ConsumerType	String

Methodology Pipeline



EXTENSION: Stacking Ensemble





Individual Classification Models

KNN K-Nearest Neighbour | Baseline comparator

⚙️ `k = 5` | `n_jobs = -1`

Non-parametric instance-based classifier. No training phase; classifies by majority vote among k nearest neighbours. Computationally expensive at prediction time on large datasets.

XGB XGBoost | Base learner (ensemble)

⚙️ `max_depth=6` | `lr=0.3` | `gbtree` | `multi:softprob`

Regularised sequential gradient boosting: $J(\alpha) = L(\alpha) + \Omega(\alpha)$, applied across both P7C and P6C scenarios. LabelEncoder was applied for P6C to ensure class labels remain continuous integers from 0, as required by XGBoost.

RF Random Forest | Base learner (ensemble)

⚙️ `100 trees` | `Gini` | `random_state=0`

Bagging-based ensemble of decision trees on bootstrap samples. Random feature subset at each split reduces variance without significant bias increase.

MLP Multi-Layer Perceptron | Base learner (ensemble)

⚙️ `100 hidden units` | `ReLU` | `Adam` | `max_iter=200`

Feedforward neural network with ReLU hidden layers and softmax output. Weights updated via backpropagation. May exhibit ConvergenceWarning at 200 iterations.

Soft Voting Ensemble: Architecture & Probability Averaging

STEP 1 — BASE MODEL OUTPUTS

For one meter reading, each model outputs a probability for every class (always sums to 1.00)

Class	RF	XGB	MLP
Normal	0.55	0.48	0.62
Theft 1	0.20	0.25	0.18
Theft 2	0.08	0.10	0.06
Theft 3	0.07	0.08	0.05
Theft 4	0.04	0.05	0.04
Theft 5	0.03	0.02	0.03
Theft 6	0.03	0.02	0.02

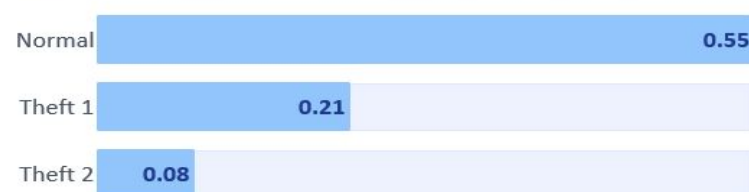
Highlighted row = winning class

STEP 2 — AVERAGE THE PROBABILITIES

$$\text{Avg(Normal)} = (0.55 + 0.48 + 0.62) / 3 = \mathbf{0.55}$$

Soft voting — equal average

Each model gets exactly 1/3 weight, always. No learning involved.



Predicted: Normal (0.55) — highest average

● XGB bad at Theft 6? Still gets 1/3 say — equal weight ignores model reliability per class

STEP 3 — FINAL DECISION RULE

Soft voting formula: $(\text{RF} + \text{XGB} + \text{MLP}) / 3$

● Simple, fast, no training needed — works well when all models are equally reliable

Soft voting formula
 $(\text{RF} + \text{XGB} + \text{MLP}) / 3$

Equal weights
Each model = 1/3 weight always

Key limitation
Weights fixed — cannot adapt per class

Base Model Performance

Accuracy (%)

Classifier	P7C	P6C
KNN	84.74%	90.87%
XGBoost	84.56%	90.58%
Random Forest	84.88%	94.71%
MLP	77.37%	84.30%
ML_Ensemble	87.30%	93.59%

Recall (%)

Classifier	P7C	P6C
KNN	84.74%	90.87%
XGBoost	84.56%	90.58%
Random Forest	84.88%	94.71%
MLP	77.37%	84.30%
ML_Ensemble	87.30%	93.59%

F1-Score (%)

Classifier	P7C	P6C
KNN	82.03%	90.43%
XGBoost	81.35%	90.11%
Random Forest	83.96%	94.57%
MLP	72.40%	81.10%
ML_Ensemble	84.33%	93.21%

Precision (%)

Classifier	P7C	P6C
KNN	80.64%	90.33%
XGBoost	80.11%	90.10%
Random Forest	83.22%	94.64%
MLP	71.65%	82.07%
ML_Ensemble	82.29%	93.62%

Meta-Learner Models

Logistic Regression

- Logistic Regression is a linear model used for classification.
- In stacking, it takes predictions (probabilities) from base models as input.
- It combines these predictions using weighted averaging.
- It applies a softmax function to output final class probabilities.

Ridge Classifier

- Ridge Classifier is a linear model with regularisation.
- It adds a penalty (L2) to prevent overfitting.
- It takes base model predictions and combines them like Logistic Regression.
- But it ensures that no single model dominates too much.

Decision Trees

- Decision Tree is a non-linear model.
- It makes decisions using if-else rules on base model predictions.
- It splits data based on conditions like:
“If XGBoost probability > 0.7 → Theft”

Extra Trees

- Extra Trees is an ensemble of multiple decision trees.
- It introduces more randomness in:
 - Feature selection
 - Split points
- Instead of finding the best split, it uses random splits.

Statistical Significance Testing: Friedman Testing

Step 1 — Friedman Test (Non-Parametric)

Purpose

Non-parametric alternative to repeated-measures ANOVA. Tests whether all 4 meta-learner candidates perform equivalently across CV folds — without assuming normality.

H_0 (Null)

All meta-learners perform equivalently across folds

H_1 (Alternative)

At least one meta-learner performs significantly differently

Scenario	χ^2 Statistic	p-value	Decision
P7C (7-class)	9.00	0.0293	Reject H_0 ✓
P6C (6-class)	8.20	0.0421	Reject H_0 ✓

Interpretation

Both p-values $< 0.05 \rightarrow H_0$ rejected in both scenarios. The observed performance ranking between meta-learners reflects genuine algorithmic differences, not fold-level noise. Selection is certifiably non-arbitrary.

Meta-Learner Selection: Candidates & Statistical Testing

P7C (7-Class): Meta-Learner Cross-Validation Results

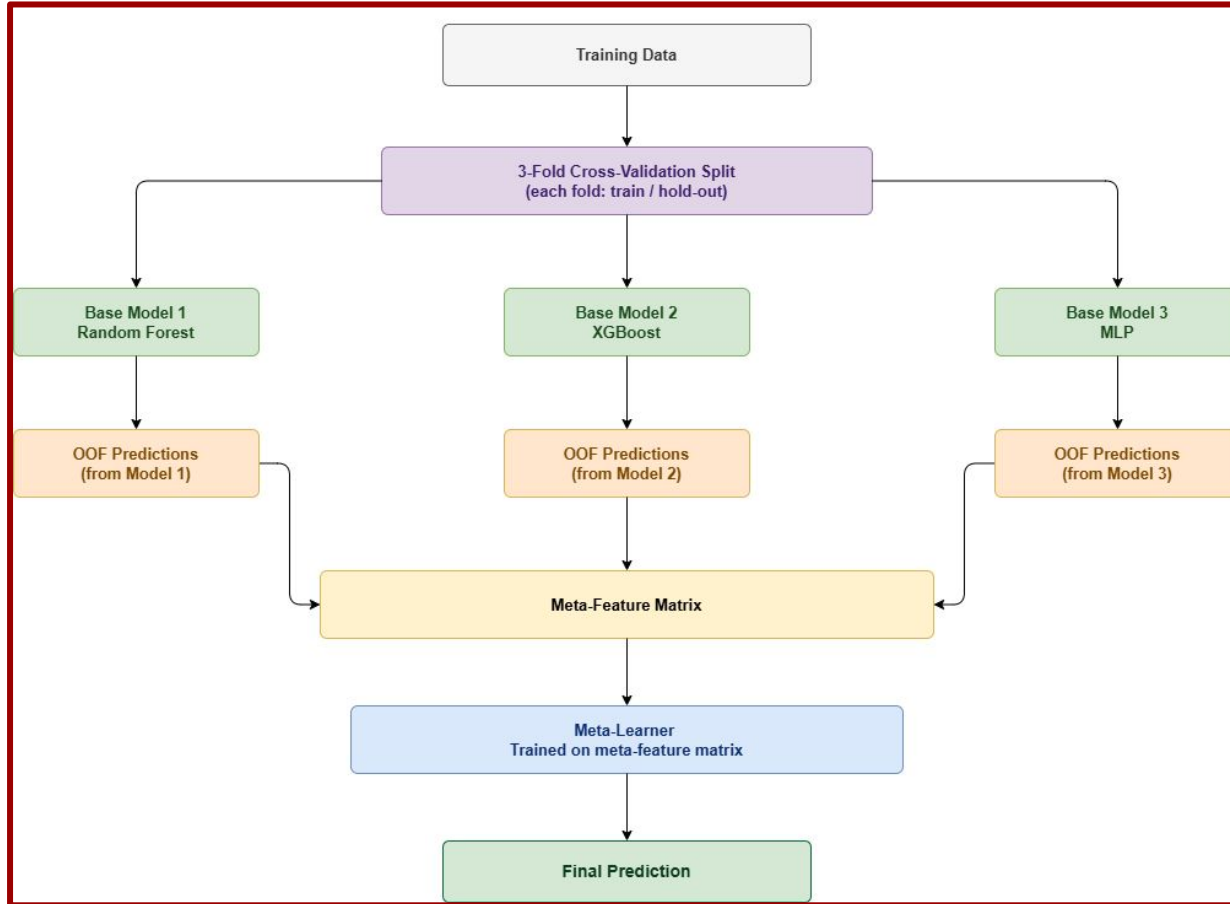
Meta-Learner	CV Accuracy	CV F1	Train Time
Logistic Regression	88.88% $\pm$ 0.04%	77.65%	13.28s
Ridge Classifier	89.02% $\pm$ 0.03%	77.32%	1.31s
Decision Tree	88.11% $\pm$ 0.03%	74.65%	8.23s
Extra Trees	89.19% $\pm$ 0.04%	78.09%	67.64s

P6C (6-Class): Meta-Learner Cross-Validation Results

Meta-Learner	CV Accuracy	CV F1	Train Time
Logistic Regression	94.80% $\pm$ 0.01%	90.22%	4.49s
Ridge Classifier	94.81% $\pm$ 0.02%	90.22%	0.87s
Decision Tree	94.58% $\pm$ 0.01%	89.92%	5.97s
Extra Trees	94.67% $\pm$ 0.03%	90.00%	33.51s

Friedman Test: P7C $\chi^2=9.00$, $p=0.0293$ (SIGNIFICANT ✓) | P6C $\chi^2=8.20$, $p=0.0421$ (SIGNIFICANT ✓) — Selection is statistically justified, not arbitrary.

Stacking Ensemble Flow Diagram



Stacking Ensemble: How It Works

STEP 1 — BASE MODEL OOF PROBABILITIES

Each base model outputs probabilities for all 7 classes via OOF cross-validation (no data leakage)

Class	RF	XGB	MLP
Normal	0.55	0.48	0.62
Theft 1	0.20	0.25	0.18
Theft 2	0.08	0.10	0.06
Theft 3	0.07	0.08	0.05
Theft 4	0.04	0.05	0.04
Theft 5	0.03	0.02	0.03
Theft 6	0.03	0.02	0.02

21 values (3 models × 7 classes) → one meta-feature row

OOF: each model predicts only on the fold it never trained on → honest predictions, zero data leakage

STEP 2 — META-LEARNER LEARNS WEIGHTS

Input: 448,000 × 21 matrix from Step 1

Compares base model predictions to true labels across all training samples

Minimises classification error → learns per-class weights for each base model

Learned weights (illustrative):

Normal class	RF: 0.28	XGB: 0.31	MLP: 0.41 ★
Theft 2 class	RF: 0.28	XGB: 0.52 ★	MLP: 0.20
Theft 6 class	RF: 0.55 ★	XGB: 0.26	MLP: 0.19

★ = highest weight for that class | Weights learned from data — not fixed

STEP 3 — FINAL DECISION RULE

$$w_1 \cdot \text{RF} + w_2 \cdot \text{XGB} + w_3 \cdot \text{MLP}$$

w_1, w_2, w_3 differ per class — learned from data in Step 2

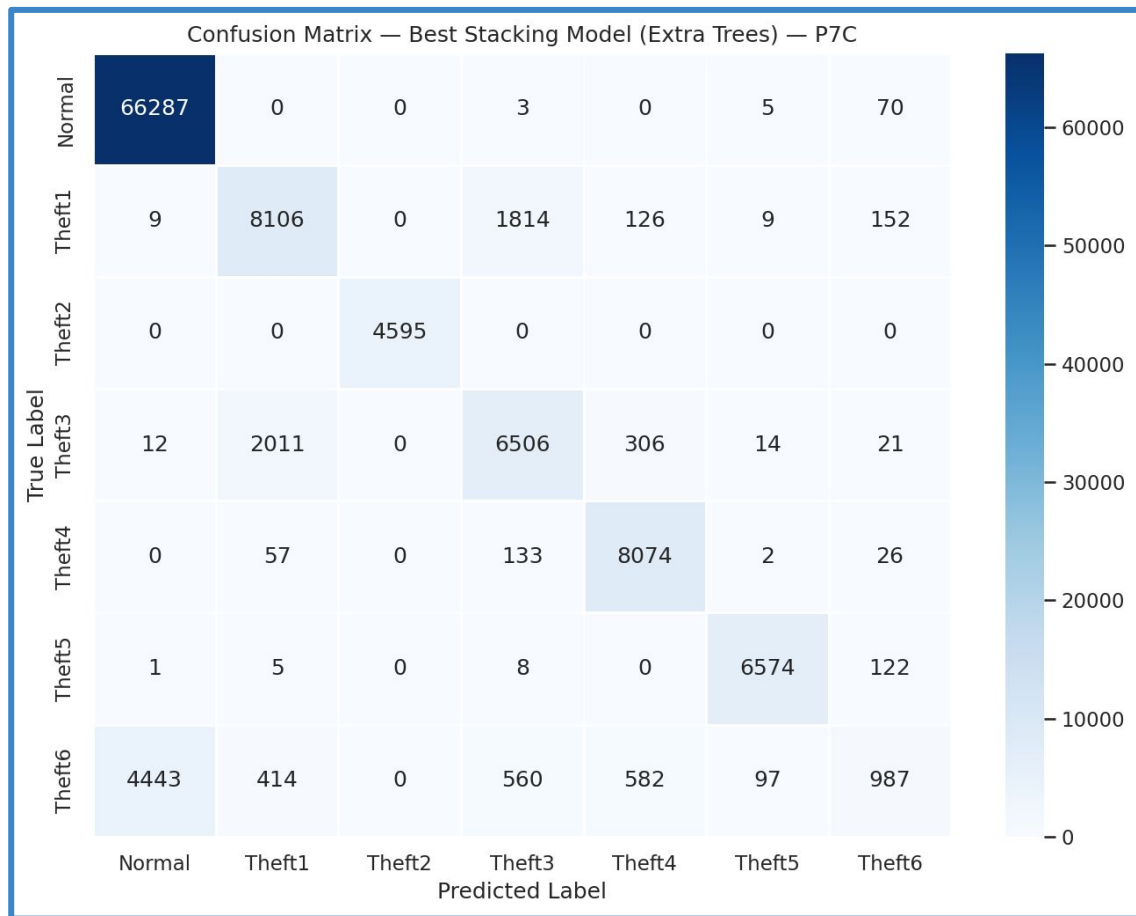
Example — Normal class:

Step 1 probs × Step 2 weights
 RF: $0.55 \times 0.28 = 0.154$
 XGB: $0.48 \times 0.31 = 0.149$
 MLP: $0.62 \times 0.41 = 0.254$

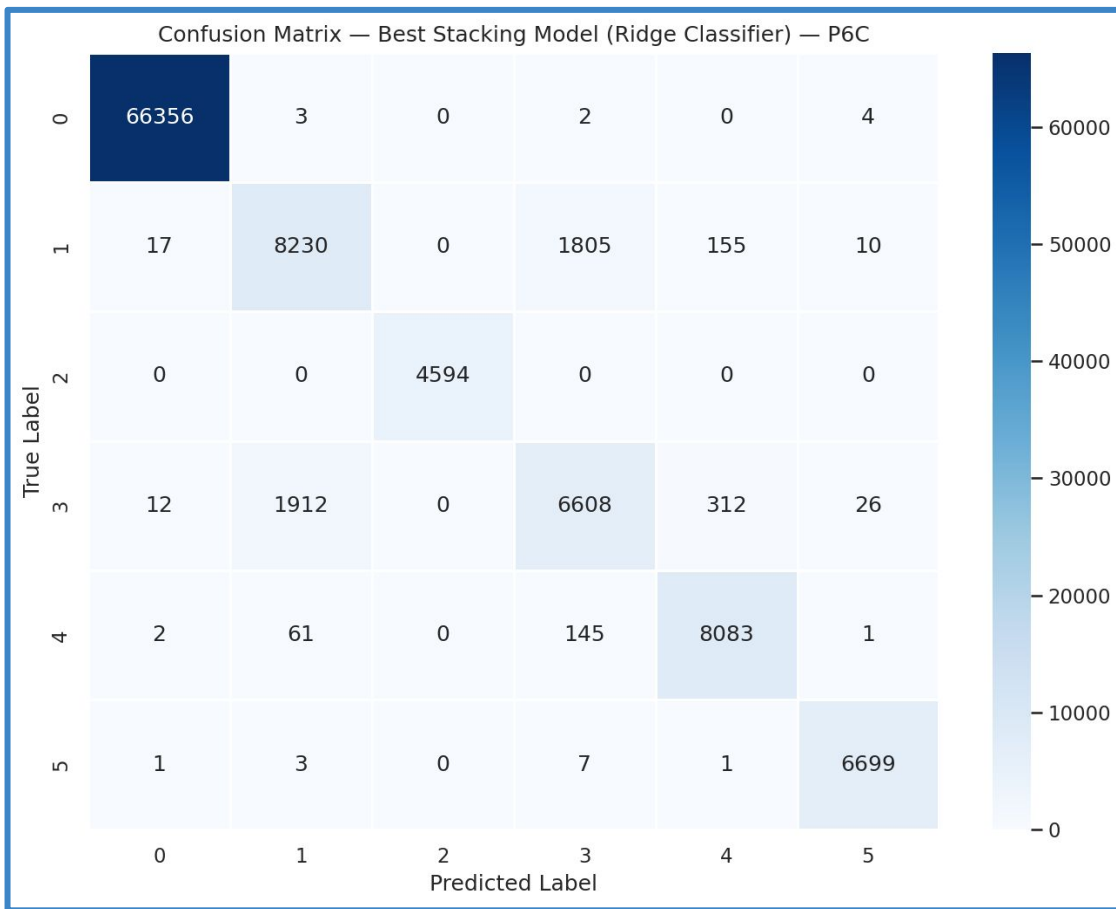
Normal score = 0.557

Predicted: Normal (highest score = 0.557)

Weights differ per class — asymmetric trust
 MLP trusted most for Normal (0.41)
 XGB trusted most for Theft 2 (0.52)



- Strong diagonal dominance → high overall classification accuracy
- Theft 2 perfectly detected (4595/4595)
- Normal class highly accurate → near-zero false negatives
- Theft 6 has highest misclassification → mostly predicted as Normal
- Reason: similar consumption pattern to Normal



- Strong diagonal dominance → high overall accuracy
- Theft 2 perfectly detected (4594/4594)
- Normal class highly accurate → near-zero false negatives
- Theft 1 and Theft 3 show mutual misclassification
- Reason: overlapping consumption patterns

Stacking Ensemble Performance

Stacking vs. Soft Voting — Key Comparison

Metric	Soft Voting P6C	Stacking P6C
Accuracy	93.59%	95.74%
Precision	93.62%	91.94%
Recall	93.59%	92.06%
F1	93.21%	91.99%
AUC-ROC	99.07%	96.80%

Metric	Soft Voting P7C	Stacking P7C
Accuracy	87.30%	90.19%
Precision	82.29%	85.84%
Recall	87.30%	80.27%
F1	84.33%	80.23%
AUC-ROC	92.39%	96.80%

Final Verdict

Stacking Ensemble

Accuracy — correctly classifies more instances overall

P7C: 90.19% | P6C: 95.74%

AUC-ROC — ranks actual thieves above normal consumers more reliably at any threshold

P7C: 96.80% | P6C: 99.52%

Precision — when a building is flagged, it is more likely to be a genuine case

P7C: 85.84% | P6C: 91.94%

Statistical validity — improvement is certified, not a product of a lucky split

Friedman test $p = 0.029$ (P7C) | $p = 0.042$ (P6C)

Soft Voting Ensemble

F1 — better balance between precision and recall across all classes

P7C: 85.49% | P6C: 94.87%

Recall — catches a higher proportion of actual theft cases overall

P7C: 88.55% | P6C: 94.88%

AUC-ROC — still strong, but lower than stacking in both scenarios

P7C: 92.39% | P6C: 99.07%

Theft6 detection — handles the most ambiguous theft class more reliably; stacking recalls only 14% of Theft6 cases

Stacking Theft6 recall: 14% | a direct consequence of class imbalance

Soft voting leads on F1 and Recall because averaging across models produces a more balanced decision boundary — particularly benefiting Theft6, the hardest class to distinguish from normal consumption. Stacking wins on accuracy and AUC in both scenarios, with a statistically certified result. In deployment, buildings are ranked by suspicion score rather than hard labels — making AUC the more operationally critical metric. Stacking is the stronger framework overall.

Binary Classification Performance (Normal vs Theft)

Stacking vs. Soft Voting — Key Comparison

Metric	Soft Voting	Stacking
Accuracy	95.80%	96.00%
Precision	99.89%	99.86%
Recall	89.80%	90.33%
F1	94.58%	94.86%
AUC-ROC	94.95%	97.86%

Limitations & Future Scope

Limitations

Single dataset (TDD2022 / OEDI): generalisability to other smart grids not assessed.

Synthetic theft data: real-world fraud may exhibit patterns not captured by the 6 algorithmic types.

No SMOTE, undersampling, or cost-sensitive learning applied.

Real-time stream-based detection not assessed — only batch classification evaluated.

Future Scope

SMOTE / ADASYN / cost-sensitive training to improve minority theft class recall (esp. Theft2).

Evaluate on P7U and P6U scenarios — unknown consumer type — to test broader applicability.

Cross-dataset evaluation on other publicly available energy theft datasets.

Real-time / incremental learning adaptations for stream-based smart meter data.

Explainability methods (SHAP values) to make predictions interpretable for utility operators.

Dedicated binary Theft6 detector as a post-processing module.

Conclusion

1

Successfully performed the soft voting ensemble on TDD2022 within acceptable performance margins.

2

Meta-learner selection via Friedman tests is statistically rigorous and certifiably non-arbitrary

3

Stacking outperforms soft voting in overall accuracy: +2.19pp (**P7C: 90.19%**) and +0.99pp (**P6C: 95.74%**), with AUC-ROC of 96.80% and 99.52% respectively.

4

Macro F1 and recall reductions are driven primarily by Theft6 (recall 14%) — a fundamental class-overlap challenge, not a stacking deficiency.

References

- [1] Mohammad, N., et al. (2023). Ensemble Learning-Based Decision Support System for Energy Theft Detection in Smart Grid Environments.
- [2] Zidi, S., et al. (2022). Theft Detection Dataset for Benchmarking and ML Classification in a Smart Grid Environment. J. King Saud Univ. DOI: 10.17632/c3c7329tjj.1
- [3] Nagi, J., et al. (2010). Nontechnical Loss Detection using SVM. IEEE Trans. Power Delivery, 25(2), 1162–1171.
- [4] Depuru, S.S.S.R., et al. (2011). Electricity Theft: Overview, Issues, Prevention and Smart Meter Approach. Energy Policy, 39(2), 1007–1015.
- [5] Jokar, P., et al. (2016). Electricity Theft Detection in AMI Using Customers' Consumption Patterns. IEEE Trans. Smart Grid, 7(1), 216–226.
- [6] Jiang, R., et al. (2014). Energy Theft Detection Using Gradient Boosting. IEEE Trans. Smart Grid, 7(5), 2410–2419.

Thank You